紫光同创 FPGA 智驾辅助系统

摘要

面向道路场景感知与辅助驾驶演示需求，我们设计并实现了一套基于紫光同创 PG2L50H FPGA 与瑞芯微 RK3568 的异构计算系统。系统支持多路摄像头和 HDMI 视频输入，由 FPGA 完成视频采集、画面组织、图像预处理及 DDR3 帧缓存，并通过 PCIe DMA 将 1280×720、RGB565 格式的视频传送至 ARM 端，形成采集、处理、感知与显示相衔接的完整链路。

FPGA 端采用流水线实现白平衡、暗通道去雾、亮度增强和双边滤波，改善前端图像质量。ARM 端利用 RGA 完成检测输入的缩放与颜色转换，在 NPU 上交替运行人员车辆和交通元素检测模型，同时由 CPU 独立完成车道边界、实虚线及横向偏移识别。决策模块结合目标位置、灯色、车道信息及观测时效，生成碰撞、行人和变道风险提示。Qt 界面叠加显示检测框、车道、局部路径和驾驶状态，并支持人工导航、停车、转向请求及模拟速度设置。采集、感知与显示采用解耦设计，通过帧缓存和结果有效性管理协调各环节运行。

板端实测表明，四路相机模式下 YOLO 推理合计约 10 fps，HDMI 单路模式约 20 fps，系统延时约 80 ms。系统实现了多源视频接入、道路感知和模拟驾驶交互，可作为嵌入式辅助驾驶视觉链路的开发验证与功能演示平台。

关键词：高速 PCIe/DMA、异构计算、辅助驾驶、目标检测、车道识别

第一部分 作品概述

1.1 功能与特性

本系统面向智慧辅助驾驶演示与验证，支持四路摄像头输入加一路 HDMI 输入；系统可完成多源视频接入、视频源选择、四宫格或单摄画面显示，并在 FPGA 端进行白平衡、去雾、亮度增强和双边滤波等图像预处理。ARM 端对人员、车辆及交通灯等目标进行检测，利用 CPU 线程并行完成车道线识别，并结合目标位置、灯色和车道关系生成碰撞、行人及变道风险提示。界面可叠加检测框、车道线、局部路径、灯色和驾驶状态，支持人工导航、停车和转向请求等模拟交互。系统将采集、感知、决策与显示解耦，便于在真实摄像头和 HDMI 场景素材之间切换，用于辅助驾驶视觉链路的联调与展示。

1.2 应用领域

系统主要面向智能驾驶视觉感知的实验验证、功能演示与嵌入式软硬件协同开发。系统支持四路摄像头真实采集加一路 HDMI 场景素材输入，可为人员车辆检测、车道线识别、交通灯判断和碰撞风险提示等算法提供可重复的视频输入，便于开展感知算法联调与驾驶规则验证。系统还可用于验证 FPGA 视频采集、图像预处理、DDR3 缓存、PCIe DMA 传输与 ARM 端 RGA 预处理、RKNN 推理、Qt 界面显示之间的接口和数据通路，适合作为异构计算平台的开发验证载体。在展示与实验教学场景中，系统能够演示多源视频切换、目标检测、车道与局部路径叠加、灯色和风险提示以及人工导航、停车和转向请求等模拟交互功能。其应用定位为辅助驾驶视觉链路的原型验证与演示平台，不替代实车安全系统或真实车辆执行控制。

1.3 主要技术特点

系统采用 FPGA 与 RK3568 异构分工：FPGA 承担多源视频采集、DDR3 帧缓存、PCIe DMA 传输及白平衡、去雾、亮度增强、双边滤波等流水线预处理；RK3568 承担 RGA 预处理、RKNN 双模型推理、传统车道识别、目标跟踪、风险决策和 Qt 显示。前后端统一采用 1280×720 RGB565 视频格式，

通过 DDR3 双槽缓存与 PCIe DMA 连接，使采集、推理和显示相互解耦，减少链路等待。ARM 端利用 RGA 完成缩放、颜色转换和 letterbox，NPU 交替执行 Vehicle 与 Traffic 模型，CPU 并行运行车道颜色、边缘和拟合流程，实现目标检测、车道识别与驾驶决策协同。决策环节结合观测代次、时效、多视角可见范围和滑窗确认管理驾驶状态，降低旧帧及模式切换对风险判断的影响。

1.4 主要性能指标

表 1-1 系统性能指标

指标	参数或实测结果
传输图像规格	1280×720@30Hz，RGB565
支持输入配置	四路摄像头加一路 HDMI
四路相机模式 YOLO 推理合计	约 10 fps
系统延时	约 80 ms
HDMI 单路模式 YOLO 推理	约 20 fps

1.5 主要创新点

(1) 异构流水分工：

FPGA 承担多源采集、DDR3 缓存、PCIe DMA 传输及像素级预处理，ARM 专注 RGA 预处理、双模型推理、车道识别与决策显示，降低单一处理器负载。

(2) 多路视频输入：

FPGA 支持四路摄像头输入加一路 HDMI 输入，满足多方向图像数据的输入。

(3) 感知决策协同：

NPU 交替调度 Vehicle 与 Traffic 模型，CPU 并行运行车道识别；决策结合多视角观测与时空信息，增强驾驶状态一致性。

(4) 交互验证闭环：

支持人工导航、停车、转向请求和模拟速度，局部路径与风险提示叠加，

形成演示闭环。

1.6 设计流程

系统按“需求与接口定义—FPGA 视频通路—跨域传输—ARM 感知决策—界面联调—板端验证”推进。先确定 1280×720 RGB565 统一格式、四路相机及两相机加 HDMI 输入配置和 FPGA 与 ARM 接口；随后搭建多路采集、DDR3 双槽缓存、视频源选择与图像预处理；打通 PCIe DMA 及 ARM 四槽接收协议；再部署 RGA 预处理、RKNN 双模型、车道识别与风险决策；最后联调 Qt 显示和人工交互，并在板端验证视频切换、目标检测、风险提示及系统延时。

第二部分 系统组成及功能说明

2.1 整体介绍

本系统由视频输入、FPGA 视频前端、PCIe 数据传输、RK3568 感知决策和显示交互组成，整体结构如图 2-1 所示。系统将连续视频采集、缓存及像素处理交由紫光同创 Logos2 系列 PG2L50H FPGA 完成，将目标检测、车道分析、驾驶规则判断与图形显示部署在 RK3568 上，形成从道路画面输入到辅助驾驶信息输出的处理链路。前后端以 1280×720、RGB565 视频帧作为统一数据接口，使视频采集与上层算法能够围绕明确的帧格式衔接。

视频输入支持四路摄像头，以及两路摄像头加一路 HDMI 的连接配置，两者共用同一套 FPGA 代码，通过引脚约束适配。摄像头用于采集实际环境画面，电脑经 HDMI 提供道路图片或视频，便于组织可重复的场景演示。多路摄像头首先进入各自的采集通道，再由缓存与读取逻辑组织输出；HDMI 经接收和格式整理进入视频源选择模块。两类输入在同一视频通路中接入后级，输出内容由画面组织和视频源选择状态决定。

在 FPGA 侧，OV5640 配置模块完成传感器初始化，采集模块根据像素时钟、行场同步和数据有效信号，将 DVP 接口的字节数据组装为 RGB565 像素。各摄像头数据经异步 FIFO 进入 DDR3 存储链路，每路使用独立地址区域和两个帧槽，通过整帧写入后发布、读取期间锁定等机制协调采集与读取。

输出端按显示模式读取完整帧，形成四宫格或单摄画面，再恢复后级所需的视频时序。HDMI 链路对接收的像素及同步信号进行寄存和格式整理，无需经过摄像头的 DDR3 帧存储环节，便可进入视频源选择模块。

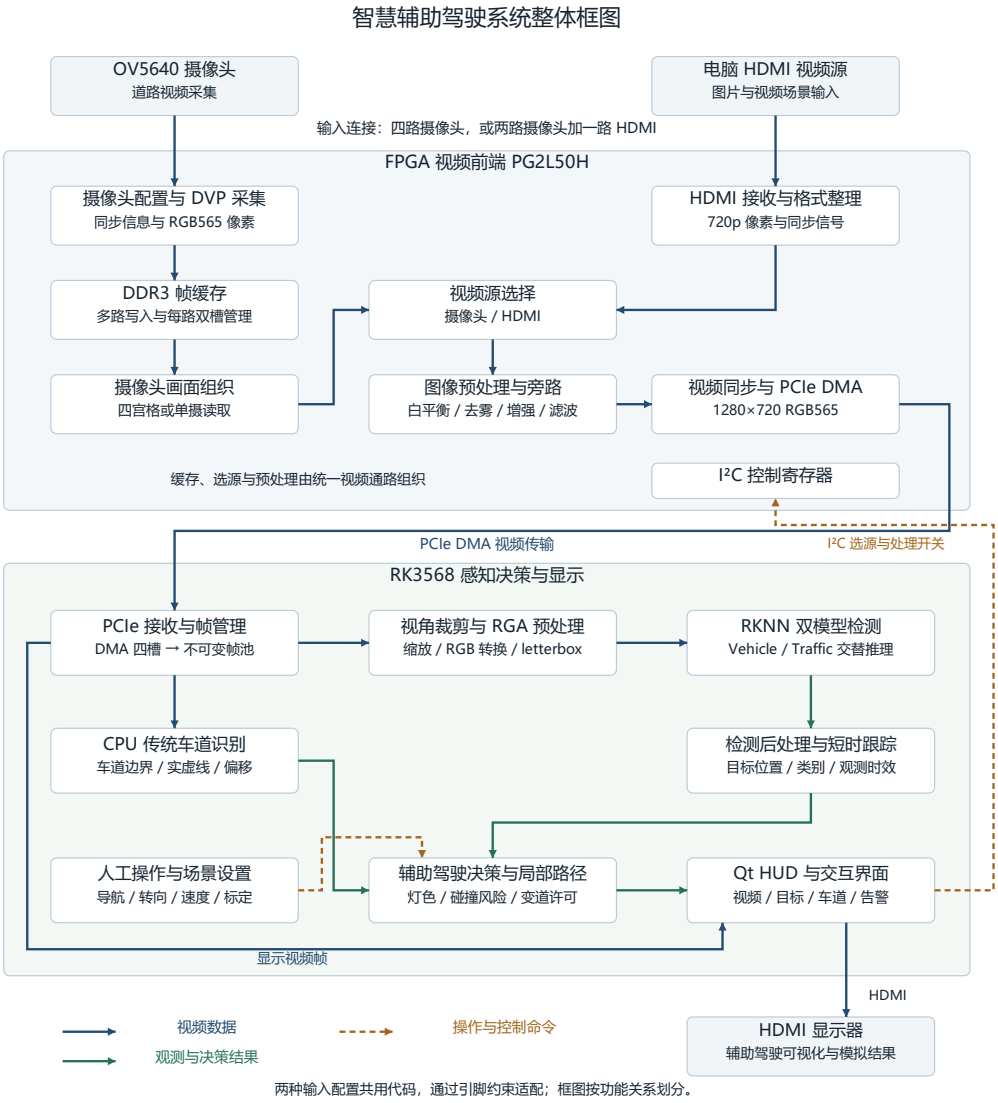


图 2-1 智驾辅助系统整体框图

视频源选择之后连接图像预处理与旁路通路。预处理模块通过白平衡、暗通道去雾、亮度增强和双边滤波等算子完成像素级处理，并使图像数据与同步信号保持对应；旁路用于输出未经该处理链路修改的视频。视频同步与 DMA 发送模块随后组织有效像素及帧边界，将图像数据写入 ARM 侧配置的接收缓冲区。这里的 DDR3 每路双槽用于 FPGA 摄像头帧存储，ARM 侧四槽 DMA 缓冲用于跨设备传输与接收，两者承担不同阶段的缓存职责，共

同衔接前端视频产生与后端处理。

RK3568 接收到帧完成信息后，由 PCIe 接收模块解析缓冲槽状态，并将有效帧纳入不可变帧池，通过最新帧投递机制向感知线程提供输入。目标检测链路按输入画面布局提取所需视角，由 RGA 完成缩放、色彩转换及 640×640 黑色填充预处理，再交给 RKNN 运行时执行 Vehicle 与 Traffic 两个模型。两个模型在同一 NPU 工作线程中交替运行，分别提供人员车辆和交通元素观测；检测结果经坐标还原、筛选和短时跟踪后送入决策模块。传统车道识别则在独立 CPU 线程中提取和拟合车道边界，给出实虚线类型、横向偏移及相关几何信息，与目标检测链路并行工作。

辅助驾驶决策模块汇集目标观测、车道结果、灯色状态和人工操作，结合数据来源、代次、时间戳及有效期判断当前信息是否可用，再形成碰撞风险、行人危险、停车约束和变道许可等状态，并生成用于显示的局部路径。四路画面下，侧向与后向目标观测参与相关安全判断；HDMI 整幅前视下，判断范围与实际可见区域相对应。标定参数用于建立前视图像与地面几何的关系，人工设置则提供导航方向、转向意图及模拟速度，使场景感知与驾驶交互在同一决策流程中衔接。

Qt 界面读取当前可显示的视频帧和最新有效结果，在画面上叠加目标框、车道边界、路径、灯色、驾驶状态与报警信息，最终经 RK3568 的 HDMI 接口输出至显示器。采集、感知和界面采用解耦方式运行，感知结果携带帧序及来源信息，场景或来源变化时通过代次隔离和状态复位处理旧结果。控制方向上，界面通过 I²C 向 FPGA 下发视频源及预处理开关，通过软件控制接口更新人工驾驶请求、标定和场景状态；FPGA 按键负责本地摄像头显示模式切换。系统由此形成视频数据前向传递、感知结果汇入决策、操作命令调节运行状态的协作关系，输出辅助驾驶提示与模拟驾驶结果。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

系统以集成紫光同创 PG2L50H FPGA 和瑞芯微 RK3568 处理器的主板为核心，外围连接摄像头、HDMI 输入模块、显示屏及 USB 键鼠。FPGA 侧

通过 FMC 接口接入一组双目摄像头，40pin 扩展口接入另一组双目摄像头或 HDMI 输入模块，构成四路摄像头、两路摄像头加一路 HDMI 两种连接配置。两者采用相同逻辑代码，通过引脚约束适配。

FPGA 与 ARM 在板内通过 PCIe 高速总线交换数据：FPGA 承担视频采集与前端处理，RK3568 运行 Linux 系统并完成感知、决策和显示。显示屏连接 ARM 侧 HDMI 输出，键盘和鼠标通过 USB 接入，用于设置参数和操作系统。

2.2.2 电路各模块介绍

硬件从总体上分为视频接入、计算与存储、板内互联、显示交互四部分，连接关系如下图所示。

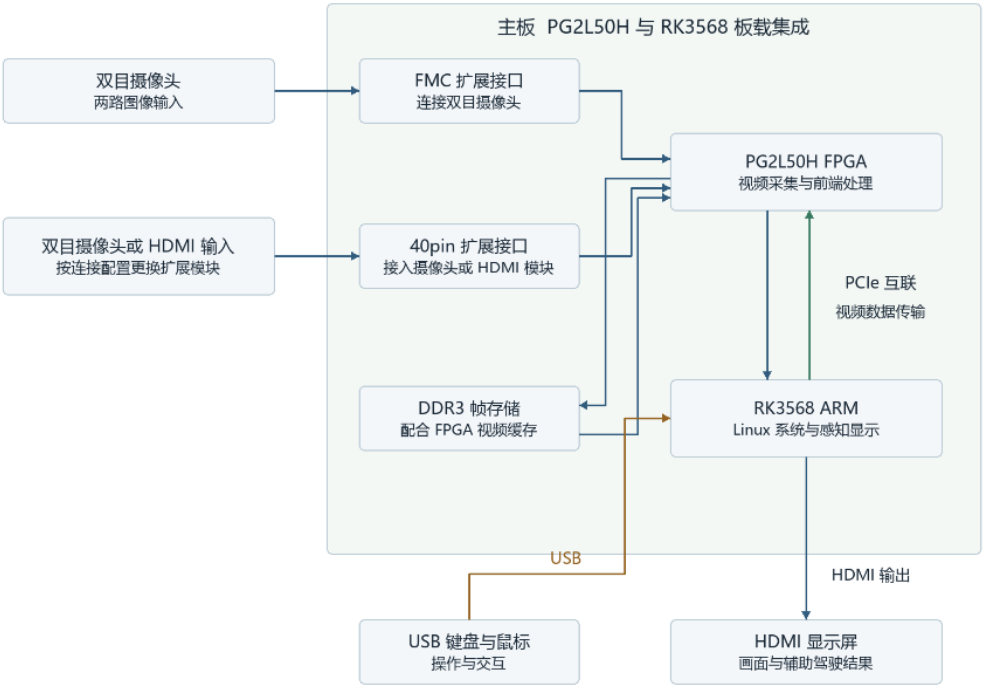


图 2-2 系统电气连接框图

2.3 软件系统介绍

2.3.1 软件整体介绍

系统的程序实现包括 FPGA 逻辑程序和 ARM 应用。FPGA 采用 Verilog 描述并行数据通路、流水线与状态机，完成视频输入配置、帧缓存调度、画面组织、图像预处理和 PCIe 发送；ARM 接收视频后完成感知、决策与显示。

两端通过统一的视频格式和控制接口协同工作。

ARM 应用运行于 RK3568 的 Ubuntu 系统，采用 Qt 和 C++17 开发，承担视频接收、道路感知、辅助驾驶决策和界面显示。目标检测通过 RKNN 运行库调用 NPU 计算，图像缩放与颜色转换由 RGA 硬件完成，传统车道识别则由 CPU 独立处理。系统提供 ADAS 与纯视频模式，分别用于感知决策演示和视频查看。

软件整体结构如下图所示。接收端将 FPGA 传来的视频保存为完整帧，分别送往目标检测、车道识别和显示通路；感知结果汇入决策模块，与人工设置共同形成驾驶状态和风险提示。采集、感知与界面分别调度，优先处理新画面，减少推理耗时引起的任务积压。结果附带来源和时间信息，切源或复位后清理旧状态，避免不同场景的结果混用。

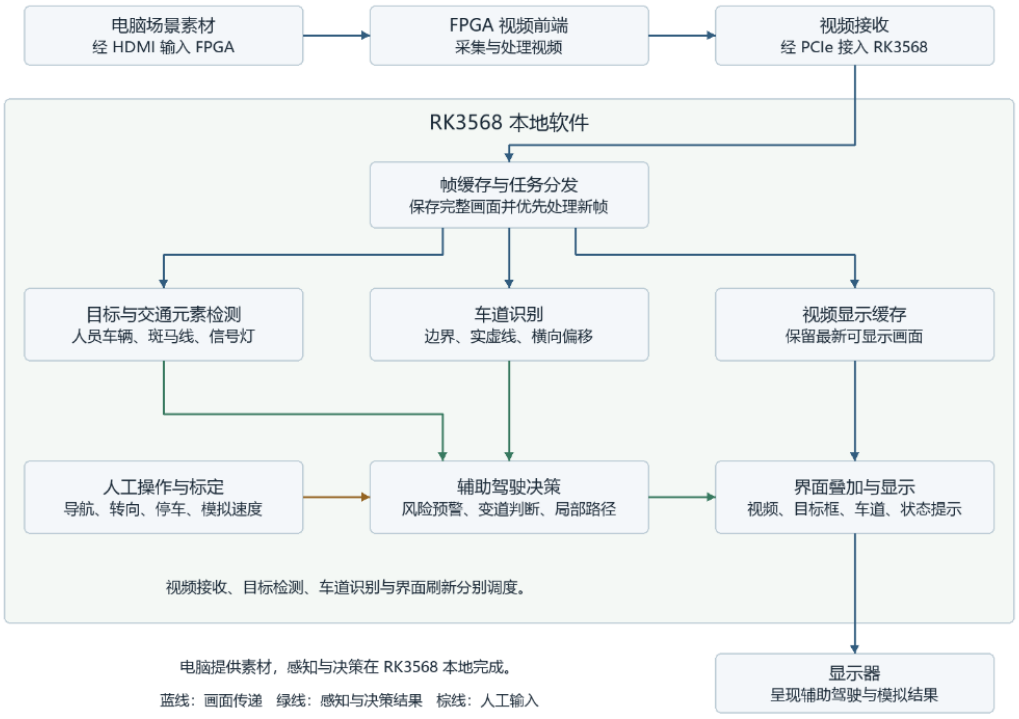


图 2-3 系统软件框图

2.3.2 软件各模块介绍

主板集成 PG2L50H FPGA 和 RK3568 处理器。FPGA 通过 FMC 连接双目摄像头，通过 40pin 扩展口接入另一组双目摄像头或 HDMI 模块，利用

DDR3 缓存摄像头画面，再经 PCIe 将视频传至 ARM。RK3568 运行 Linux，通过 HDMI 连接显示屏、USB 连接键鼠，并通过 I²C 控制 FPGA 的视频源和预处理开关。



图 2-4 系统各模块框图

2.3.2.1 FPGA 视频输入与帧缓存调度

输入部分由器件配置和像素采集两类逻辑组成。上电后，配置逻辑通过串行接口向摄像头及 HDMI 接收芯片写入参数，使输入格式与后续通路一致。摄像头输出的一个 RGB565 像素由连续两个字节构成，采集逻辑在像素时钟下完成高低字节组装，同时生成与完整像素对应的数据有效标志。行场同步用于确定图像边界，避免将消隐期间的数据写入画面。

多路摄像头分别工作在各自的采集时钟下，不能直接与 DDR3 访问逻辑共用节拍。输入像素先经异步 FIFO 缓冲并打包为较宽的数据字，再由存储时钟域按突发方式写入 DDR3。突发访问将零散像素集中传输，减少命令开销；共享端口在多路写入和输出读取之间仲裁，使采集与读出能够交错进行。

各路摄像头使用独立地址区域和两个帧槽。写侧只有在整帧数据完整且

写入结束后才发布该槽，读侧选择已完成帧并在读取期间锁定，防止一幅输出画面混入正在更新的数据。没有可写槽或出现采集溢出时，逻辑放弃本次不完整输入，保护已有完整帧，并从后续帧边界重新建立写入状态。

读侧通过输出 FIFO 衔接 DDR3 突发返回与固定光栅节奏，先准备数据再输出像素。发生欠流时，将本帧剩余区域置黑并排空迟到数据，等待后续帧恢复，避免把延迟返回的像素误当成下一帧。采集溢出、写入丢帧和输出欠流分别保留状态，便于区分异常来源。

HDMI 输入由接收芯片解码后送入 FPGA，颜色数据、同步与有效信号在接收像素时钟下同时寄存，再转换为统一的 RGB565 格式。

摄像头与 HDMI 在视频源选择处汇合，为后级提供统一的视频接口。

2.3.2.2 FPGA 画面组织与图像预处理

摄像头输出支持四宫格和单摄模式。四宫格读取逻辑在输出帧开始时记录各路可用帧槽，将每幅 1280×720 源图按隔行、隔列采样整理为 640×360 子图，放入对应象限；单摄模式则读取所选通道的完整画面。读取会话中保持源帧不变，缺少有效源帧的区域填黑。帧槽锁定保证各路读取期间画面完整，并不代表多路摄像头同时曝光。该过程完成多视角画面组织，不包含俯瞰视角变换或重叠区域融合。

预处理入口将 RGB565 扩展为 RGB888，依次连接白平衡、暗通道去雾、亮度增强和双边滤波。白平衡采用灰度世界思路，统计有效图像中三个颜色通道的均值，以公共灰度均值计算通道修正比例，再调整像素颜色。统计与像素校正分阶段进行，倒数查表配合定点乘法减少逐像素除法，输出超出范围时作饱和处理。

去雾模块先提取每个像素的最小颜色分量，再通过邻域最小值获得暗通道，结合大气光估计计算透射率，并恢复场景亮度与颜色。透射率设置下限，避免较小分母放大量化误差；原始颜色与透射率经过延迟对齐后共同参与恢复。窗口运算通过行缓存和比较逻辑连续完成，适合视频流逐像素输入。

亮度增强在 YCbCr 空间中进行，主要调整亮度分量。模块计算局部平均亮度，据此查表取得增益，使暗区域得到较强补偿、亮区域增益受限；色度

分量延迟后与增强结果重新组合。后级双边滤波同时考虑像素空间位置和亮度差异，利用亮度生成的权重对亮度、色度作加权处理，抑制噪声并保留边缘。

2.3.2.3 FPGA 数据传输与运行控制

发送逻辑以既有 PCIe DMA 框架为基础，将视频载荷接入事务发送通路。ARM 下发启动请求后，FPGA 等待完整帧边界再接收待发送像素，避免从半帧开始计数。视频侧与 PCIe 侧之间设置异步 FIFO，分别承担像素写入和宽数据读取，用于适配时钟差异及链路短暂停顿。

DMA 按行组织请求。1280 个 RGB565 像素对应 2560 字节，目标地址随完成的行传输递增；一行可根据 PCIe 最大载荷限制拆成多个写事务，整行完成后再推进计数。累计 720 行形成一帧，随后发布完成状态并切换主机接收槽位。下游暂时不能接收时，发送端依照握手与流控等待，不以请求已经发出代替传输完成。

ARM 预先配置四个接收缓冲区，FPGA 按槽位轮换写入，并返回完成帧所在的索引。图像载荷通过 DMA 传输，启停、地址和状态通过控制寄存器交换；视频源与预处理开关则由原有 I²C 接口控制，摄像头显示模式可通过本地按键切换。相关逻辑分层组织，使视频帧、行传输和事务完成相互对应，主要流程如下图。

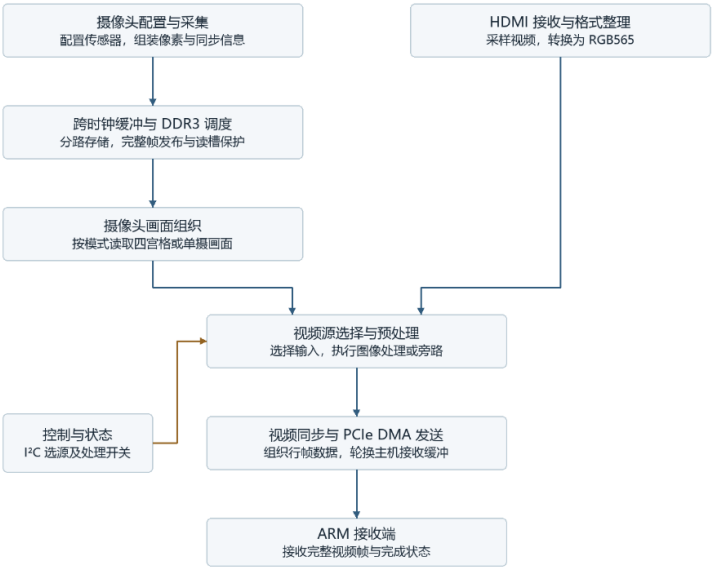


图 2-5 数据传输流程图

2.3.2.4 视频接收与帧管理

ARM 接收模块读取 PCIe 帧状态，确认有新帧后，根据槽位索引定位图像。有效像素复制到应用拥有的帧缓存，再以只读方式供检测、车道与显示共享。这样，驱动缓冲继续接收下一帧时，不会覆盖算法正在使用的画面。

每帧附带来源、序号、接收时间和布局信息，用于选择视角并关联感知结果。尚未处理的任务可以被新帧替换，正在处理的帧则保留至使用结束；应用缓存繁忙时跳过本次处理输入，避免等待持续积累。切源或复位后更新场景身份并清理旧结果，停止采集时先结束后台任务，再释放相关资源。

2.3.2.5 目标检测与持续追踪

检测模块按输入布局提取整幅前视或四宫格视角，由 RGA 完成裁剪、等比例缩放与颜色转换，生成 640×640 的模型输入，空余区域填黑。缩放比例与填充位置用于推理后的坐标恢复，使检测框能够准确叠加到原图。模型载入时检查输入输出规格及类别约定，避免权重与解析方式不一致。

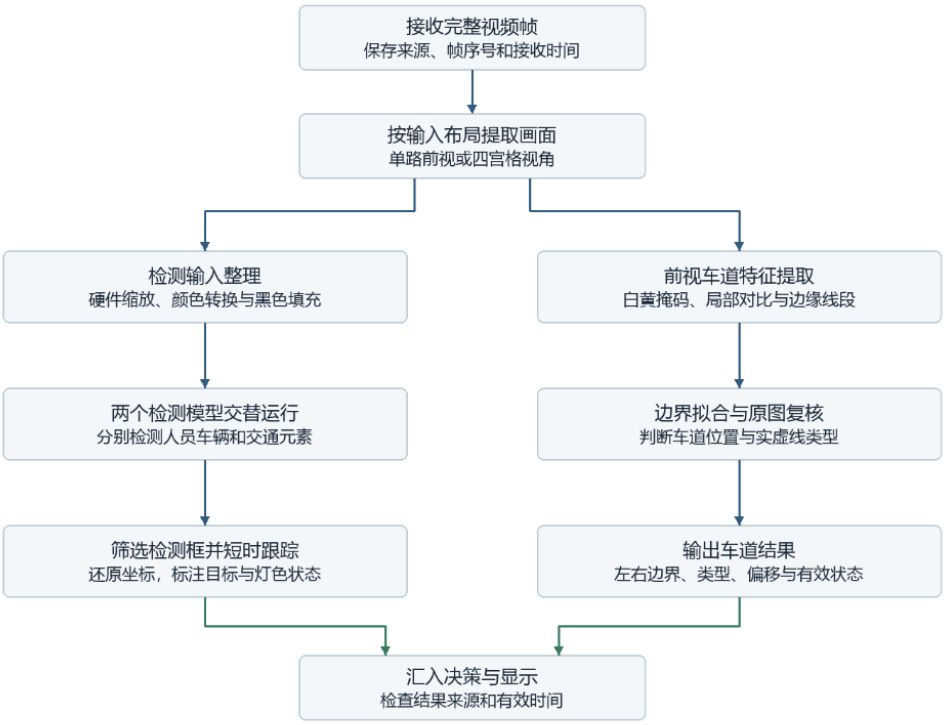


图 2-6 感知支路框图

人员车辆模型与交通元素模型在同一 NPU 线程中交替执行，分别提供主要人员车辆观测，以及斑马线、行人和灯色信息。前视具有较高处理优先级，出现转向意图后增加目标侧的调度机会。推理后通过置信度筛选和重复框抑制保留目标，再恢复原图坐标；灯色依据前视及对应区域组织，不由侧后视结果决定通行状态。

追踪按视角与类别关联连续目标，利用运动趋势补偿短暂遮挡或漏检，预测框单独标识。预测不增加真实确认次数，也不作为新的测距或危险解除依据。模型缺失、处理失败与正常空检测分别记录，使决策能够区分感知不可用和有效观测中没有目标，感知支路如图 2-6 所示。

2.3.2.6 车道识别

车道模块在独立 CPU 线程中处理前视画面，先利用白黄颜色、局部对比和边缘提取候选线段，再根据位置与方向区分左右边界并完成拟合。道路宽度、交点和边界位置用于排除明显不合理的组合，避免背景线条直接形成车道结论。

边界确定后，在原位置附近复核漆段的连续性与间断情况，判断实线、虚线或未知状态。输出包含边界位置、可见范围、置信度和归一化横向偏移。当条件不足时，保留未知或无效状态，并决策限制相关动作，保证驾驶安全。

2.3.2.7 辅助驾驶决策与路径规划

决策模块汇集目标、灯色、车道和人工请求，首先检查结果是否属于当前场景、是否仍在有效期内，再形成停车、行人危险、碰撞风险与变道判断。灯色和车道类型通过连续观测确认，降低单次波动的影响；已有危险也需要新的有效观测才能解除，不能因模型失败或结果过期就转为安全。

变道需要同时满足虚线许可、道路边界、前方路径与停车约束。四路模式还检查目标侧观测是否新鲜、晚于本次请求且无相关阻挡；单路模式只评价前视可见路径。局部规划沿车道生成 16 个图像坐标点，用平滑横向过渡表示模拟变道，过渡受阻时回退为保持车道或停车。

操作者提供导航方向、转向意图、模拟速度和限速，系统结合这些输入

显示驾驶建议。前视标定通过地面四点及实测尺寸建立图像与地面的关系，只有来源、尺寸及距离条件有效时才提供米制目标距离。路径和驾驶状态判定的流程如图所示。

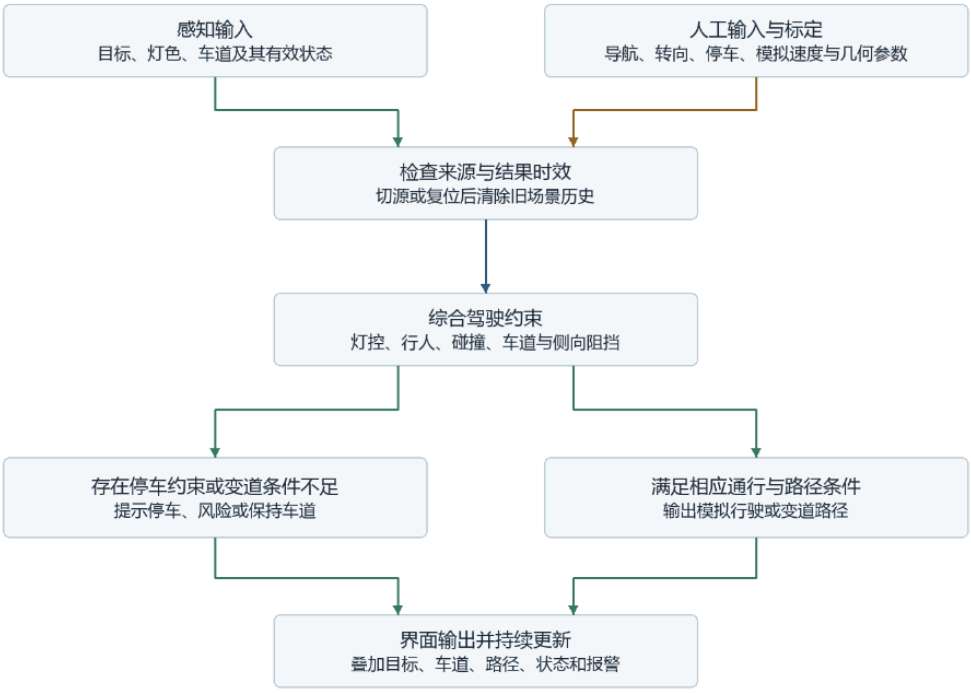


图 2-7 辅助驾驶与决策流程图

2.3.3 软件各模块连接情况

各模块的主要输入、输出如下表。

表 2-1 系统各模块输入输出表

软件模块	主要输入	主要输出
FPGA 视频输入与缓存	摄像头字节流、HDMI 像素及同步信号	摄像头完整帧缓存、可供选择的视频流。
FPGA 画面组织与预处理	完整帧、显示模式及处理开关	四宫格或单摄画面、处理后或旁路像素流。
FPGA 传输与控制	有效视频、ARM 缓冲地址及控制命令	DMA 视频数据、帧完成与槽位状态。
视频接收与帧管理	PCIe 视频数据、帧完成状态、输入布局	完整图像帧及来源、帧序和接收时间。

目标检测与跟踪	图像帧、视角、模型、转向意图	目标位置、类别、灯色、跟踪标识与处理状态。
车道识别	前视画面	车道边界、实虚线类型、相对偏移及有效状态。
辅助驾驶决策	感知结果、人工请求、模拟速度与限速	停车约束、风险报警、变道许可及局部路径。
界面与统计	视频帧、感知与决策结果	叠加画面、状态提示、运行统计及截图报告。

第三部分　完成情况及性能参数

3.1 整体介绍

本系统已形成以 PG2L50H FPGA 和 RK3568 为核心的辅助驾驶演示平台，将视频接入、前端处理、PCIe 传输、道路感知与显示交互连接起来。FMC 侧接入双目摄像头，40pin 扩展口接入摄像头或 HDMI 模块；运行时由 ARM 侧连接显示屏和 USB 键鼠，用于操作与上位机显示。



图 3-1　系统整体图

3.2 工程成果

3.2.1 电路成果

电路成果为主板、扩展板及摄像头模块的组合装配。FMC 扩展板与主板插接，40pin 侧安装摄像头或 HDMI 模块。

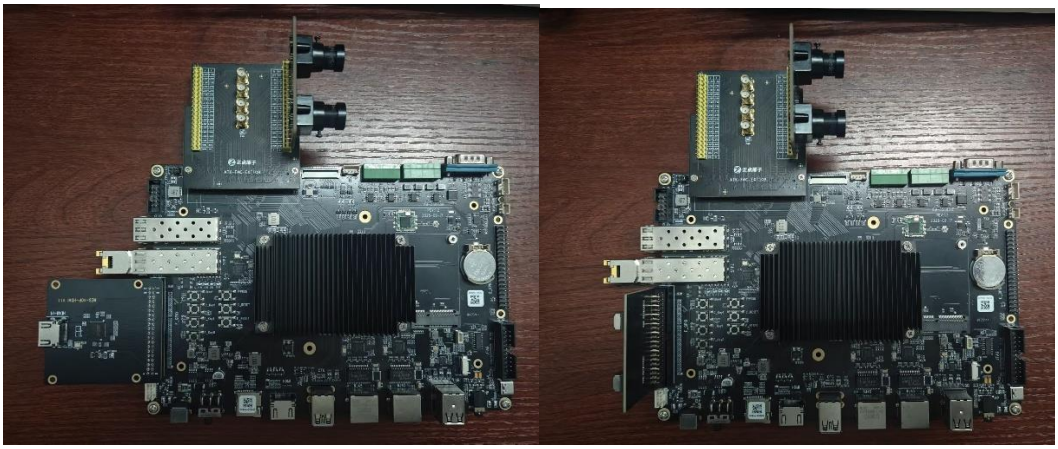


图 3-2 板卡连接图



图 3-3 转接板模块图

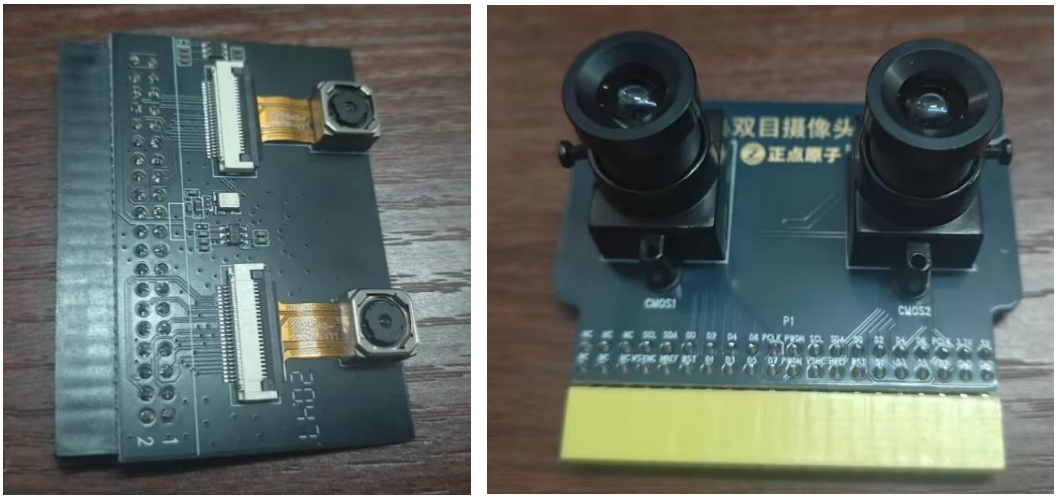


图 3-4 摄像头模块图

3.2.2 软件成果

软件已形成 ADAS 与纯视频两种工作模式。

ADAS 界面能够叠加人员车辆、交通元素、车道边界、局部路径及报警信息，并提供导航、转向、停车请求和模拟速度设置；纯视频模式用于查看输入画面。前视放大与四宫格显示相配合，便于观察道路细节和多视角状态。



图 3-5 软件界面展示图

3.3 特性成果

3.3.1 视频接入

系统支持四路摄像头，以及两路摄像头加一路 HDMI 输入，统一通过 1280×720@30Hz、RGB565 视频格式接入 ARM 处理链路。

3.3.2 图像预处理

FPGA 能够按设置执行白平衡、去雾、亮度增强和滤波，具体测试情况如下图 3-6 所示。

（1）首先展示白平衡图像处理算法：



图 3-6 白平衡前后比对图

可以看到与原始图像相比，经过白平衡补偿模块的图像的偏色得到改善，白平衡算法成功实现。

（2）接下来展示去雾算法：



图 3-7 去雾前后比对图

可以看到与原始图像相比，经过去雾算法的图像明显更清晰，雾气被成功去除。

（3）接下来展示亮度增强算法：



图 3-8 增亮前后比对图

可以看到与原始图像相比，经过光线增强的图像明显更亮，光线增强模块成功。

3.3.2 道路感知与辅助判断

依托 ARM 侧的 NPU 推理加速以及 CPU 调度，实现了对复杂交通场景的感知与预警。

（1）正常情况下，车辆按照交通指示行驶。



图 3-9 车辆正常形式示意图

（2）右变道且车道为虚线时，交通系统正常指示右转。

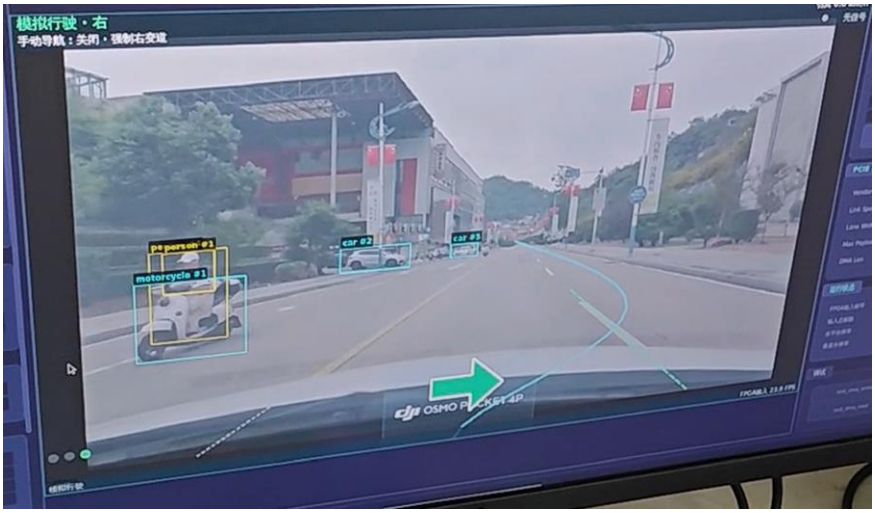


图 3-10 车辆右转线路规划图

（3）左变道且车道为实线时，交通系统显示实现强制变道。

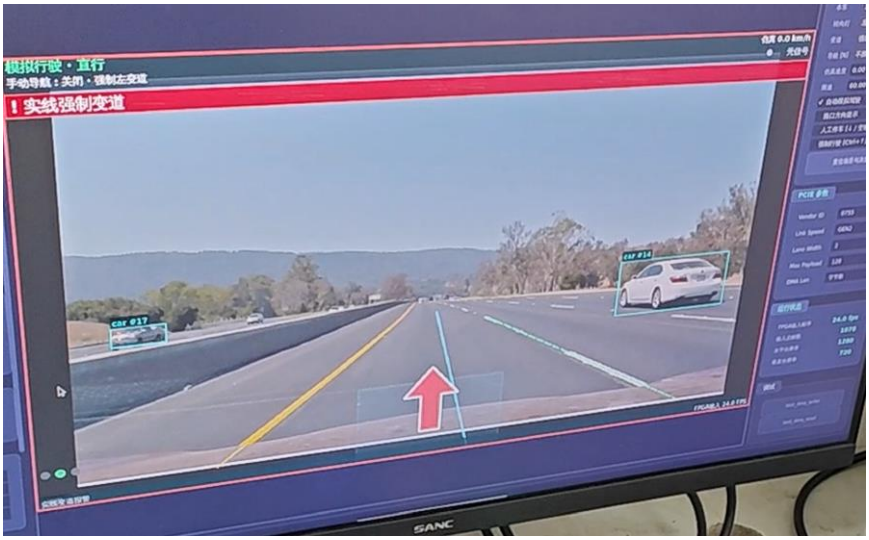


图 3-11 车辆强制变道警告图

（4）前方有行人在斑马线，系统显示行人急刹。



图 3-12 系统检测到行人并警告

（5）离前方车辆过近，且速度较快，存在碰撞风险，系统显示碰撞危险。



图 3-13 车辆碰撞预警提示图

（6）车辆闯红灯时，显示红灯强行驶。



图 3-14 车辆违反交通信号灯系统警报

3.3.3 性能参数

板端实测中，四路相机模式下 YOLO 推理合计约 10 fps，HDMI 单路模式约 20 fps，系统延时约 80 ms。多视角任务需要分配检测处理时间，因此四路与单路的推理频率不同。主要规格与性能如下。

表 3-1 系统板端实测性能表

项目	参数或实测结果	说明

视频传输格式	1280×720，RGB565	FPGA 与 ARM 之间的统一图像规格。
视频输入配置	四路摄像头；两路摄像头加一路 HDMI	共用逻辑代码，通过引脚约束适配。
四路相机模式 YOLO 推理合计	约 10 fps	前视全量检测，其他三路抽样检测。
HDMI 单路模式 YOLO 推理	约 20 fps	对整幅前视画面进行检测处理。
系统延时	约 80 ms	系统实测延时。

第四部分　总结

4.1　可扩展之处

后续可利用 FMC 接口的引脚资源，将四路摄像头统一接入 FMC，保留 40pin 扩展口用于 HDMI 输入，形成四路摄像头与一路 HDMI 的接入配置。在图像处理方面，可通过相机标定、畸变校正、透视映射和重叠区域融合，将四路画面转换为统一的环视俯瞰图。

计算架构方面，可探索将计算密集性功能从 ARM 端卸载到 FPGA 端。通过脉动阵列，将 YOLO 推理迁移至 FPGA，通过模型量化和算子复用适配逻辑资源，使 ARM 主要承担后处理、辅助驾驶决策与界面显示，并进一步优化两端的数据交互与系统整体延迟。

4.2　心得体会

从摄像头画面的接入，到 FPGA 处理、PCIe 传输，再到 ARM 端识别和显示，本项目让我们对嵌入式视觉系统的开发有了更完整的认识。整机能否正常运行，取决于各个模块能否与前后环节配合。图像格式、同步信号和缓存状态中的细节，都会影响最终呈现的画面与识别结果。在把这些环节连接起来的过程中，我们逐渐学会沿着数据流理解系统，关注问题出现的位置及其对后续处理的影响。课堂上的时序、缓存和并行处理也因此有了具体的应用，使我们对 FPGA 与 ARM 各自承担的任务，以及两者如何协同工作，有了更深入的理解。

这项工作也让我们体会到，团队协作需要建立在共同理解系统的基础上。硬件连接、FPGA 逻辑和 ARM 应用之间联系紧密，开发时既要完成各自的任务，

也要清楚自己的输出将如何被下一环节使用。明确接口和运行条件，有助于减少联调中的理解偏差，让问题更容易定位。对于辅助驾驶这样的应用，我们也更加重视结果的使用边界：画面中能够识别目标，还需要结合当前场景和信息是否有效，才能给出恰当的提示。完成项目的收获，既包括具体功能的实现，也包括逐渐形成从整体观察问题、用实际结果验证设计的习惯。后续完善系统时，我们希望继续保持这种工作方式，把已有功能做得更稳定，让演示和操作更加清楚、方便。

第五部分 参考文献

[1] 瑞芯微电子股份有限公司. RKNN-Toolkit2[CP/OL]. [2026-11-03]. https://github.com/airockchip/rknn-toolkit2.

[2] 瑞芯微电子股份有限公司. RKNN Model Zoo: YOLOv5[CP/OL]. [2026-11-03]. https://github.com/airockchip/rknn_model_zoo/tree/bad6c7334531becaf90a561988519b7bec34d0ab/examples/yolov5.

[3] JOCHER G. YOLOv5 by Ultralytics: version 7.0[CP/OL]. (2022-11-22)[2026-11-03].https://github.com/ultralytics/yolov5/tree/v7.0.

[4] LIN T Y, MAIRE M, BELONGIE S, et al. Microsoft COCO: common objects in context[C]//Computer Vision—ECCV 2014. Cham: Springer, 2014: 740-755. DOI: 10.1007/978-3-319-10602-1_48.

[5] BUCHSBAUM G. A spatial processor model for object colour perception[J]. Journal of the Franklin Institute, 1980, 310(1): 1-26. DOI: 10.1016/0016-0032(80)90058-7.

[6] HE K, SUN J, TANG X. Single image haze removal using dark channel prior[C]//2009 IEEE Conference on Computer Vision and Pattern Recognition. Piscataway: IEEE, 2009: 1956-1963. DOI: 10.1109/CVPR.2009.5206515.

[7] TOMASI C, MANDUCHI R. Bilateral filtering for gray and color images[C]//Sixth International Conference on Computer Vision. [S.l.]: Narosa Publishing House, 1998: 839-846. DOI: 10.1109/ICCV.1998.710815.

[8] CANNY J. A computational approach to edge detection[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986, 8(6): 679-698. DOI: 10.1109/TPAMI.1986.4767851.

[9] DUDA R O, HART P E. Use of the Hough transformation to detect lines and curves in pictures[J]. Communications of the ACM, 1972, 15(1): 11-15. DOI: 10.1145/361237.361242.

[10] THE QT COMPANY. Qt 5.15 documentation[EB/OL]. [2026-11-03]. https://doc.qt.io/archives/qt-5.15/index.html.

[11] OPENCV. OpenCV: open source computer vision library: version 4.10.0[CP/OL]. [2026-11-03]. https://github.com/opencv/opencv/tree/4.10.0.